

State deformation from word embedding to LLM (and back) for next-token learning

Let $\text{Emb}: \text{Token} \rightarrow \text{Vector}$ be a word embedding, and $v_i = \text{Emb}(t_i)$.
Denote a LLM by a map $\text{Token} \rightarrow \text{Continuation}$.

Let $\theta = \{ (W_1, b_1), (W_2, b_2) \}$ be parameters
for a **feedforward network** defined by
 $\text{FFN}_\theta(h_i) := \text{ReLU}(h_i W_1 + b_1) W_2 + b_2$

A **state machine** (S, O) begins in an initial state h_0 ,
that gets updated with each successive input t_i ,
via $h_i = S(h_{i-1}, t_i)$ where S is a transition function, and
then outputs $o_{i+1} = O(h_i)$, where O is an output function.
(The right index shift is for autoregressive continuation.)
Diagrammatically, picture a state machine as a spine
with vertebrae hidden states h and ribs either side t - h - o ,
token to state to output.

implicitRNN functions similarly to a vanilla RNN,
processing sequence inputs one by one. For each
time step i , where $t_i \in \mathbb{R}^p$ represents the i -th token in a
sequence $(t_1, t_2, \dots, t_m)$, the input u_i for implicitRNN is a
concatenation of t_i and the previous hidden state h_{i-1} ,
akin to a vanilla RNN. The implicit prediction rule for
implicitRNN is as follows:

$$h_0 = 0, \quad k_0 = 0$$

$$u_i = [t_i \ h_{i-1}]'$$

$$k_i = \phi(Ak_i + Bu_i) \text{ (equilibrium equation)}$$

$$h_i = S(h_{i-1}, t_i) := Ck_i + Du_i \text{ (prediction equation)}$$

where $A := \Gamma - \text{diag}(|\Gamma| \mathbf{1}) + \gamma \mathbf{Id}$, $0 < \gamma < 1$,
ensures A well-posed for ϕ , that is,
the fixed point equation has a solution and it is unique.

In this setup, the recurrent layer is replaced with an

implicit structure comprising the equilibrium and prediction equations, where we learn matrices A, B, C, D and fixed points k_i . Note that h_i is a function of k_i .

Alice

Is a state machine with transition and output functions (S, O) , where S is defined by the implicitRNN above, and state h_i is viewed as a contextual representation of token t_i . Given a token sequence $t_1, t_2, \dots, t_i$ the goal is to find a token continuation o_{i+1} with certain properties. The state machine output enables this by choosing the continuation o_{i+1} to be $O(h_i)$, defined by attention layers with the desired (perplexity and Renyi α -entropy) properties.

Attention layer

Input h_i .

Let score be a desired similarity function, like a reproducing kernel.

For $i \in [m]$, define probabilities

$\alpha_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i, k_j)$.

Let $h_i^\wedge = \sum_{j \in [i]} \alpha_{ij} v_j$, where $v_j = \text{Emb}(t_j)$,

and define update $h_i' := \text{FFN}\theta(h_i^\wedge)$.

Output h_i' .

Stack L attention layers $\Theta = (\theta_1, \dots, \theta_L)$ composing updates and now let h_i' denote the final L -th update.

For $i \in [m]$, define probabilities

$\beta_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i', k_j)$

and let $j^*i := \arg \max \text{ over } j \in [i], \beta_{ij}$.

Define token continuation $o_{i+1} = O(h_i) := t_{j^*i}$,

the token whose fixed point k to date is most similar (probable under distribution β) to update h' .

Define the average Renyi α -entropy of $\{t_1, \dots, t_m\}$, a measure of semantic alignment by:

$E(\{t_i\}) := (1/m) \sum_i H(i, \alpha)$,

where $H(i, \alpha) := H_\alpha(\beta_{ij})$, the Renyi α -entropy of $\{\beta_{ij}\}$.

Together perplexity of output **O** and average Renyi α -entropy **E** train the h_i to holistically capture syntax and semantics.

Alice training problem

Let $\lambda > 0$, learn implicitRNN matrices A, B, C, D and fixed points k_i for h_i , as well as, feedforward network parameters Θ to minimize:

$$[\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})]$$

Remarks:

O next-token continuation $t_1, \dots, t_i$, o_{i+1} reflects the word embedding through the $h_i^\wedge$.

Perplexity intrinsically evaluates the continuation, regularized by average Renyi α -entropy.

Solving the Alice training problem, is solving the **next-token learning** problem jointly constrained by syntactic co-occurrence (perplexity) and semantic alignment (Renyi α -entropy).

Thus an Alice state h_i results in output $o(h_i)$, a token continuation, that defines a LLM.

Ecila

is a sister state machine **(S, O')**, where $o'_i = \mathbf{O}'(h_i) := \text{FFN}\theta'(h_i)$ approximates $v_i := \text{Emb}(t_i)$

In detail, the **Ecila training problem**

Learn implicitRNN matrices A, B, C, D and fixed points k_i for h_i , and feedforward network parameters θ' for o'_i to minimize:

$$\sum_{i \in [m]} 1 - \text{Cos}(o'_i, v_i)$$

So **Ecila** simulates Emb, the word embedding.

AliceEcila_τ

Is the state machine **(S, O, O')** with two output functions,
mother to embedding Ecila and transformer sister Alice.

The **Alice**ecila_A_τ training problem

Given $\lambda > 0$, for $\tau \in [0,1]$ learn matrices A, B, C, D and fixed points k_i for h_i ,
parameters Θ and θ' , to minimize:

$$(1 - \tau) \sum_{i \in \{m\}} 1 - \text{Cos}(o_i, v_i) + \tau [\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})]$$

So 'word embedding' Ecila(S, O') is deformed by Aliceecila(S, O, O')_τ
to 'LLM' Alice(S,O), in the sense Aliceecila_τ states h_i deform from
Ecila's states to Alice's states.

This also defines a map from Word Embedding to LLM,
given by (Emb, Alice).

And back

Given Alice(S, O), a function of Emb, $o_{i+1} := O(h_i)$.

Note that the function, t_i to Emb(o_{i+1}) defines a new embedding Emb',
and (Alice, Emb') defines a map LLM to Word Embedding.

Unrolling this generates a chain: Emb, Alice, Emb', Alice', Emb'', Alice'', ...

From the purposes of token continuation, we have many choices,
which Emb-Alice pair iterate, and which $\tau \in [0,1]$ for Aliceecila_τ
whose output O determines a token continuation.

Thus Aliceecila_τ is a fifth strategy for learning continuations.